3DWEBSOFT INC. FOUNDATION

FUTURE TECH SCHOOL PROGRAM

Faculty Name :

Course Duration : 5 Months

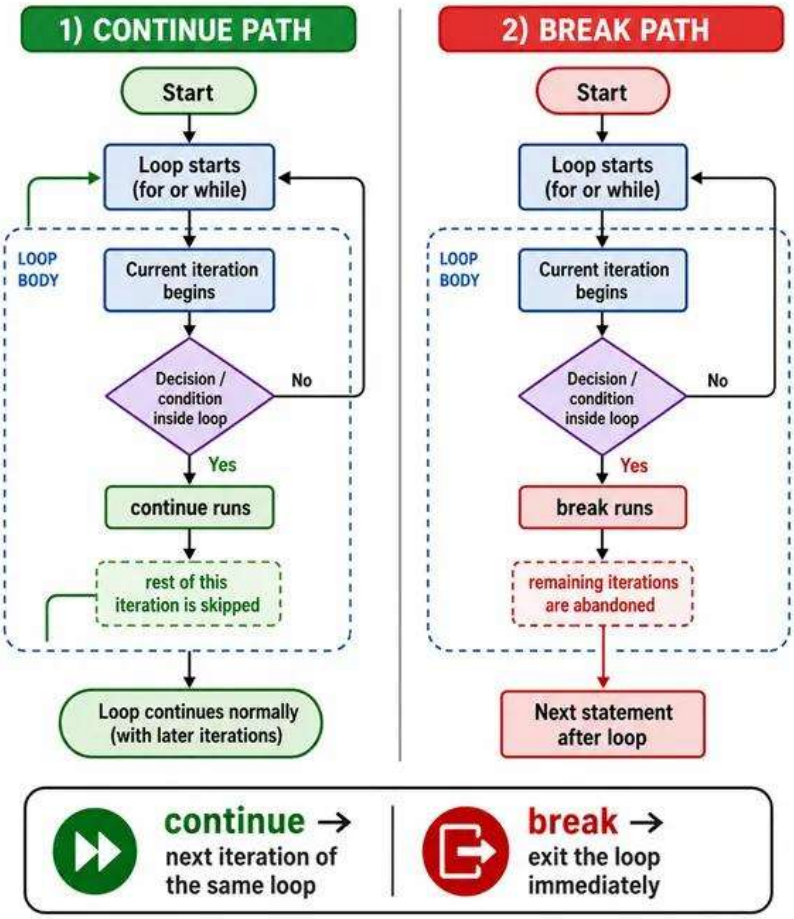
Classes : Offline & Online

Daily Teaching : 2 Hours

Total Classes : 100

Level 2 – Young Technologist (7th–8th Standard)

Lesson 24: Break & Continue in Python



Learning Objectives

By the end of this lesson, students will be able to:

- ✅ Understand the purpose of **break** and **continue** statements.
- ✅ Learn how break stops a loop.
- ✅ Learn how continue skips one iteration of a loop.
- ✅ Use break and continue in for and while loops.
- ✅ Solve simple real-life problems using loop control statements.

What are Break and Continue?

Sometimes, while using a loop, we want to:

- 🛑 **Stop the loop immediately**, or
- ⏭️ **Skip one step and continue with the next repetition.**

Python provides two special statements for this:

- **break** – Stops the loop completely.
- **continue** – Skips the current iteration and moves to the next one.

Real-Life Example

Imagine you are reading a storybook.

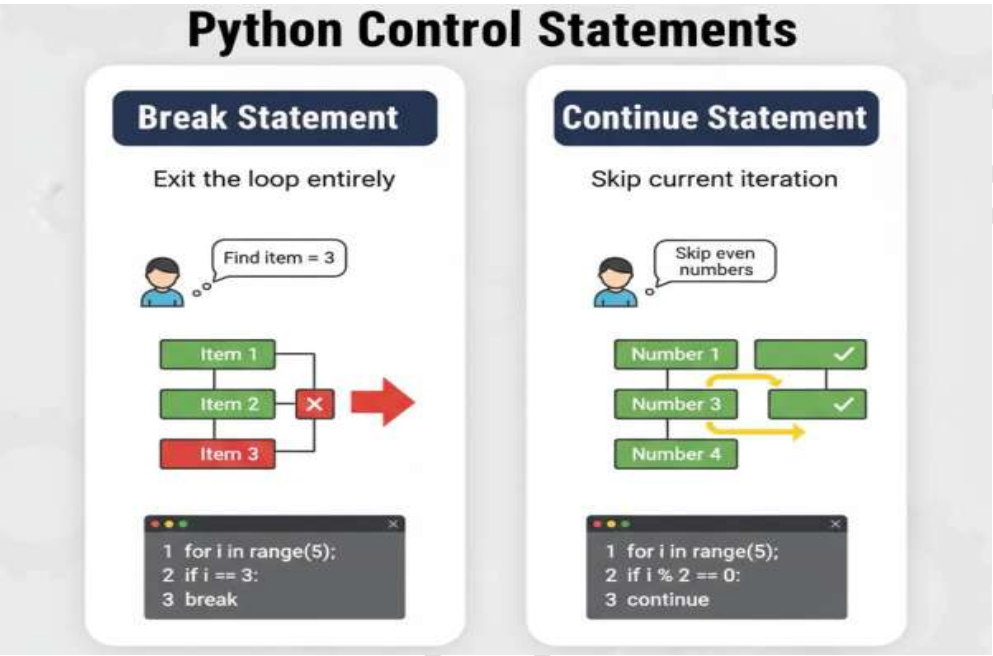
- 📖 You read every page.
- 🚪 If your teacher says **"Stop reading!"**, you stop immediately.
  → This is like **break**.

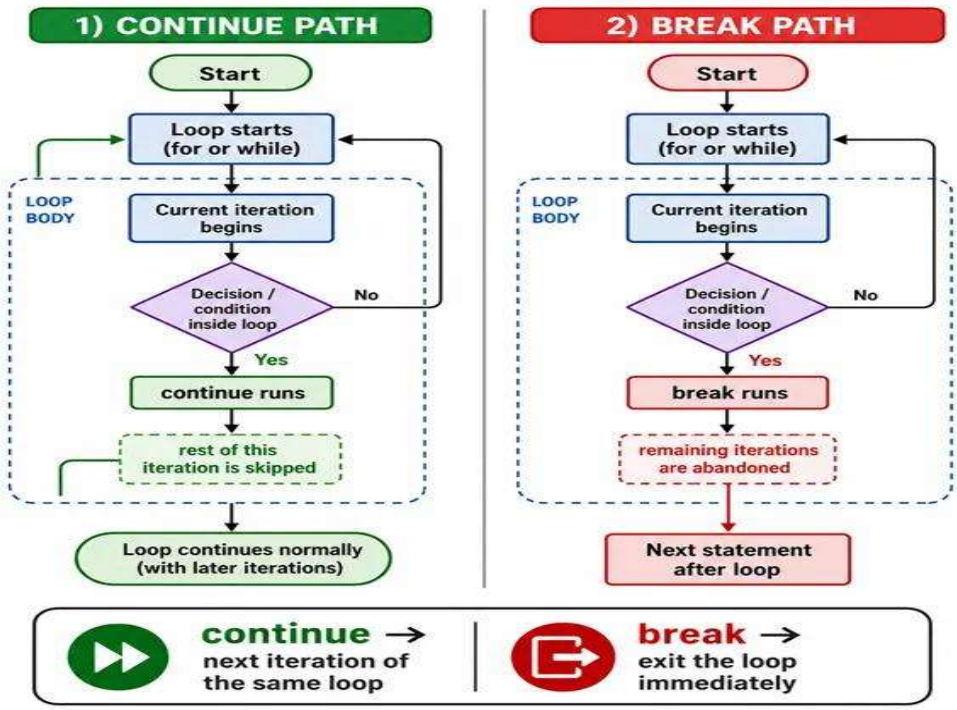
Now imagine:

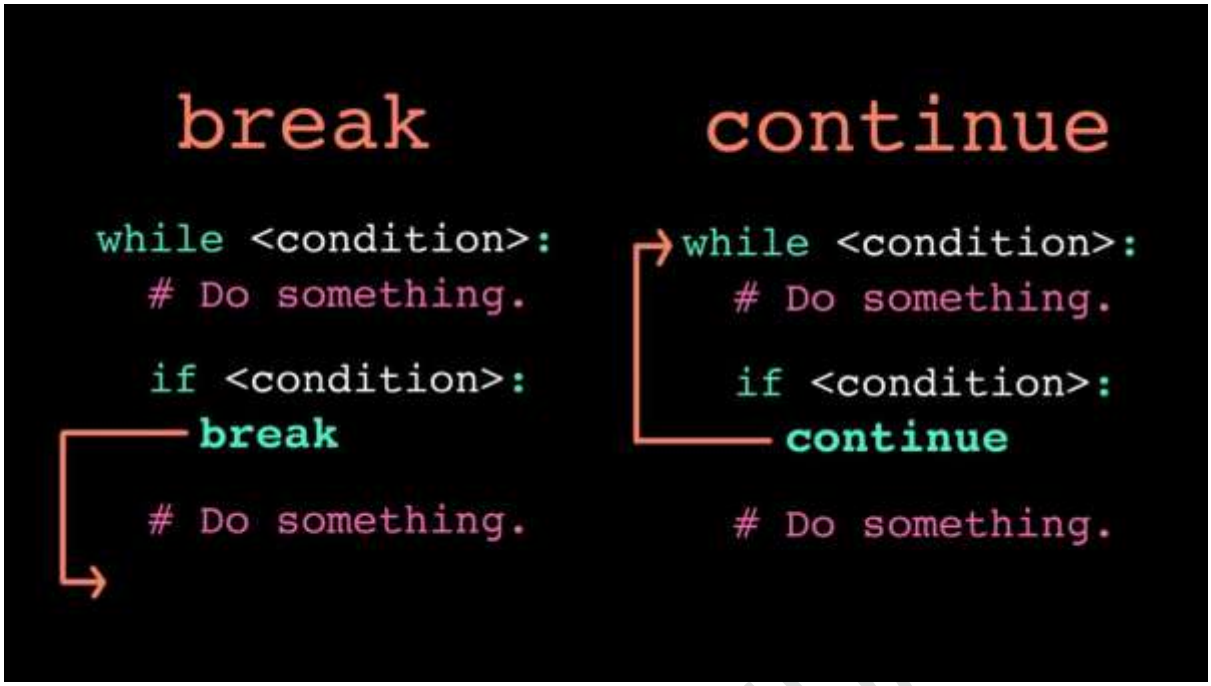
3DWEBSOFT FOUNDATION

- 📖 You skip **Page 5** because it is damaged and continue reading from **Page 6**.
  → This is like **continue**.

Break vs Continue







What is break?

The **break** statement **immediately stops the loop**.

Once Python reaches break, it exits the loop.

Syntax

```
for variable in range(...):
    if condition:
        break
```

Example 1: Break in a For Loop

```
for i in range(1, 11):
    if i == 6:
        break
    print(i)
```

Output

3DWEBSOFT FOUND

1

2

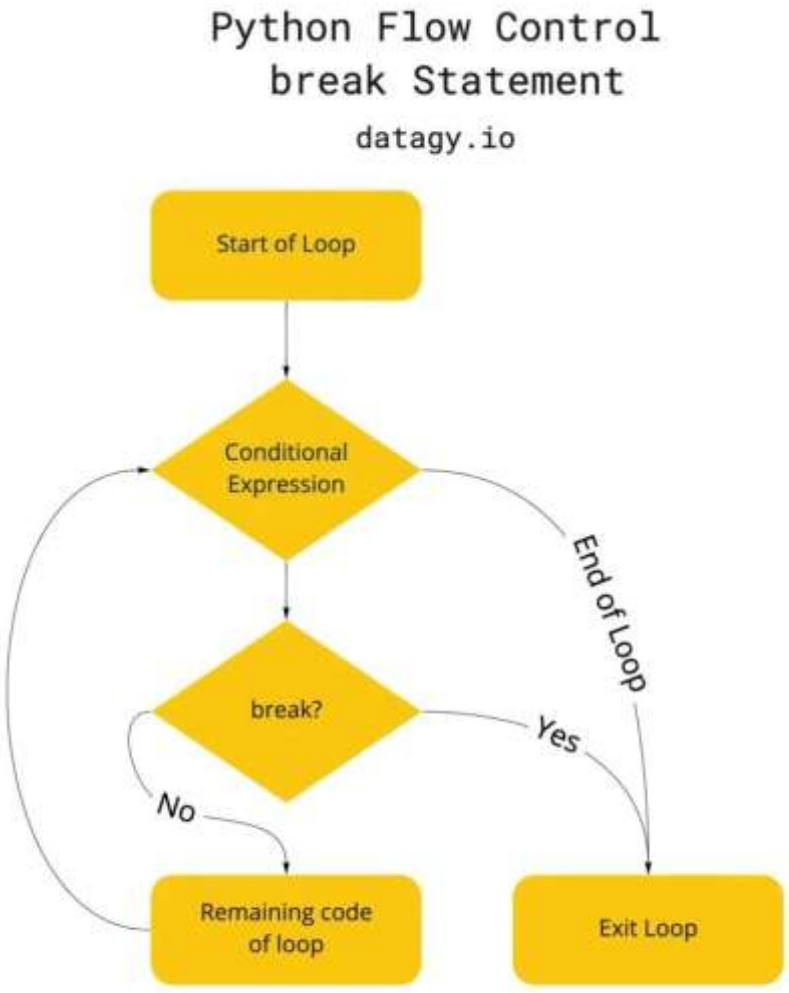
3

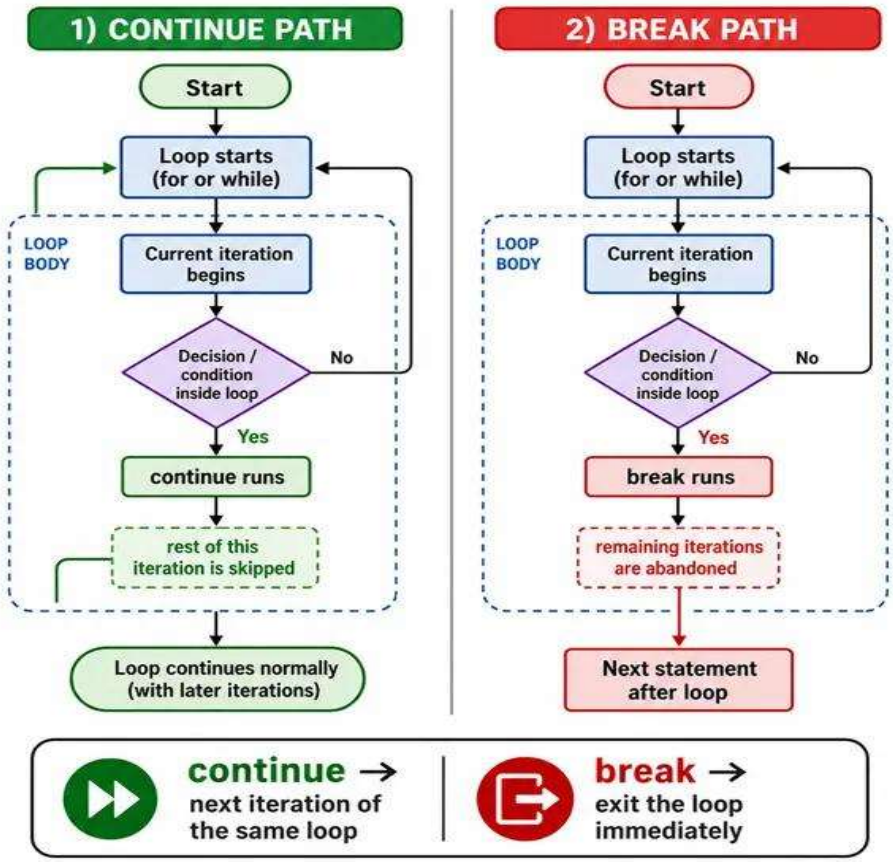
4

5

The loop stops when i becomes **6**.

Break Flowchart




1) CONTINUE PATH
Start
Loop starts (for or while)
LOOP BODY
Current iteration begins
Decision / condition inside loop
No
Yes
continue runs
rest of this iteration is skipped
Loop continues normally (with later iterations)
2) BREAK PATH
Start
Loop starts (for or while)
LOOP BODY
Current iteration begins
Decision / condition inside loop
No
Yes
break runs
remaining iterations are abandoned
Next statement after loop
continue → next iteration of the same loop
break → exit the loop immediately


unstop
Break And Continue Statements In Python
START
Continue
Break
Execute Next Iteration
Exit The Loop

Example 2: Break in a While Loop

```
count = 1


while True:
    print(count)


    if count == 5:
        break


    count += 1
```

Output

1

2

3

4

5

What is continue?

The **continue** statement **skips the current iteration** and moves to the next repetition of the loop.

The loop does **not stop**.

Syntax

```
for variable in range(...):
```

2DWEBSOFT FOUNDATION

```
if condition:
    continue
```

Example 1: Continue in a For Loop

```
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
```

Output

1

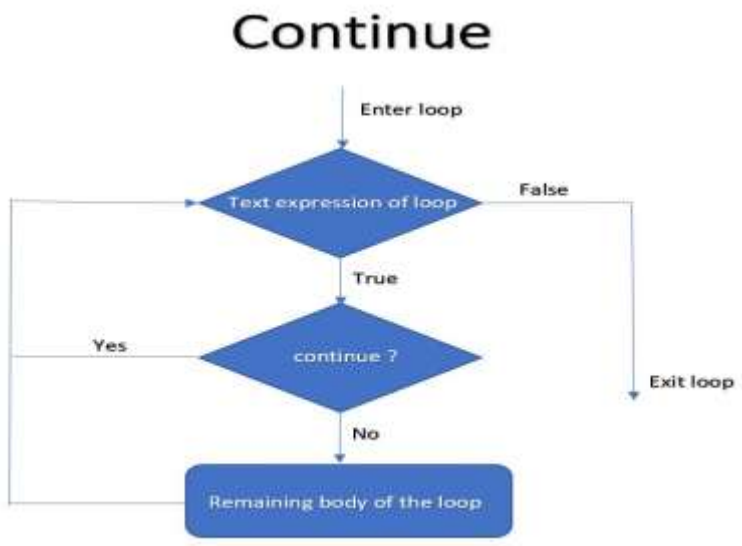
2

4

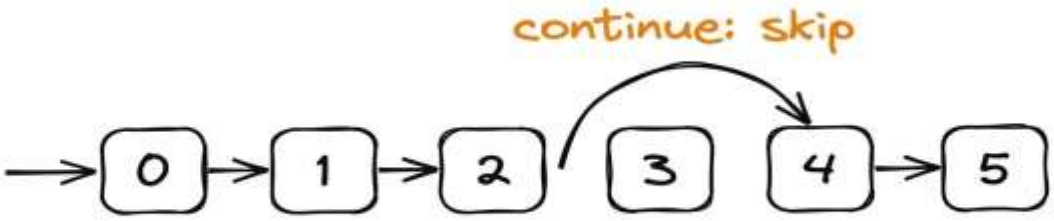
5

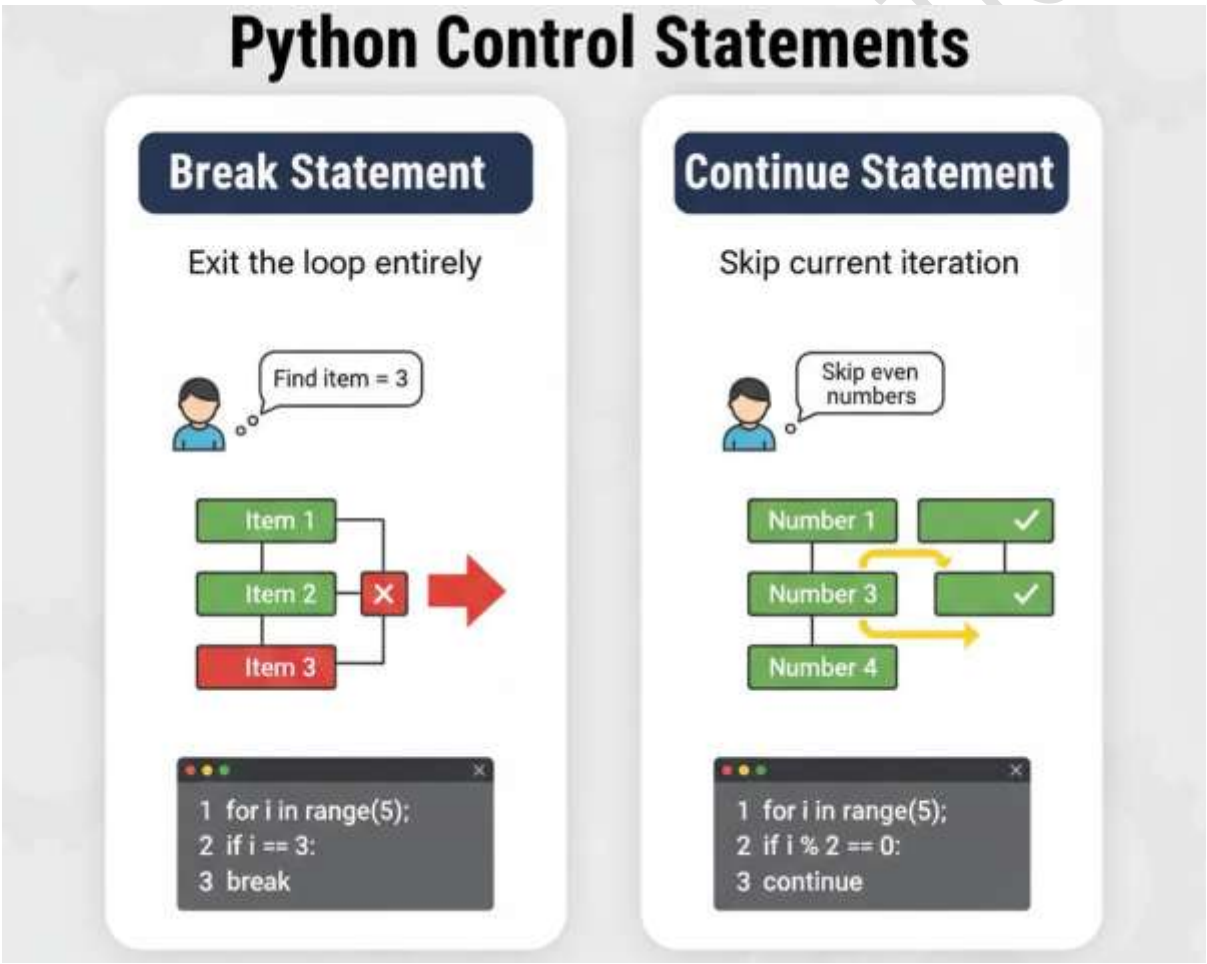
Number **3** is skipped.

Continue Flowchart



3D CYBERSOFT FOUNDATION





Example 2: Continue in a While Loop

count = 0

```
while count < 5:
    count += 1

    if count == 3:
        continue

    print(count)
```

Output

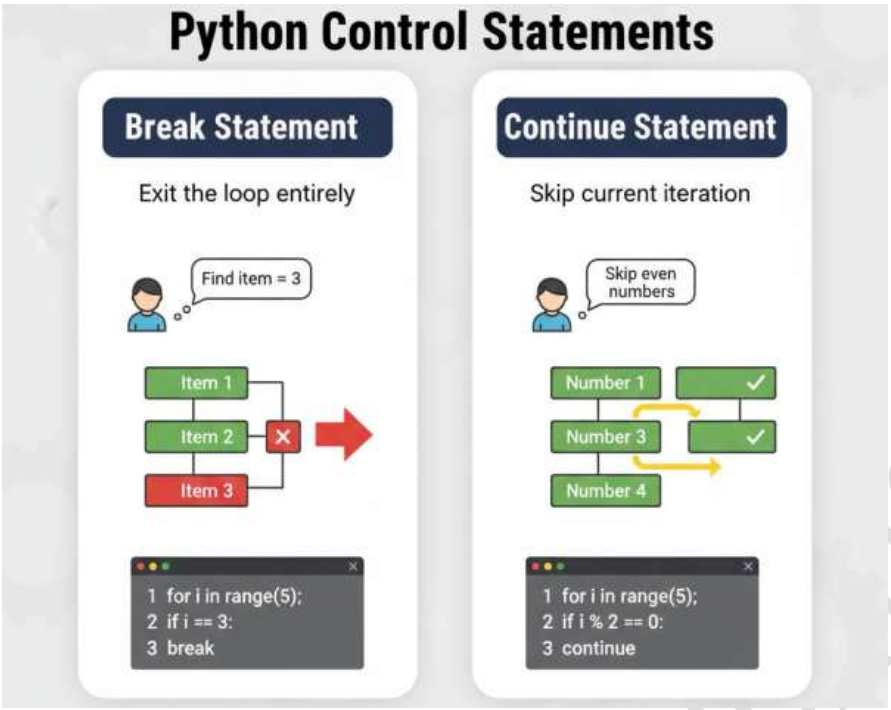
```
1
2
4
5
```

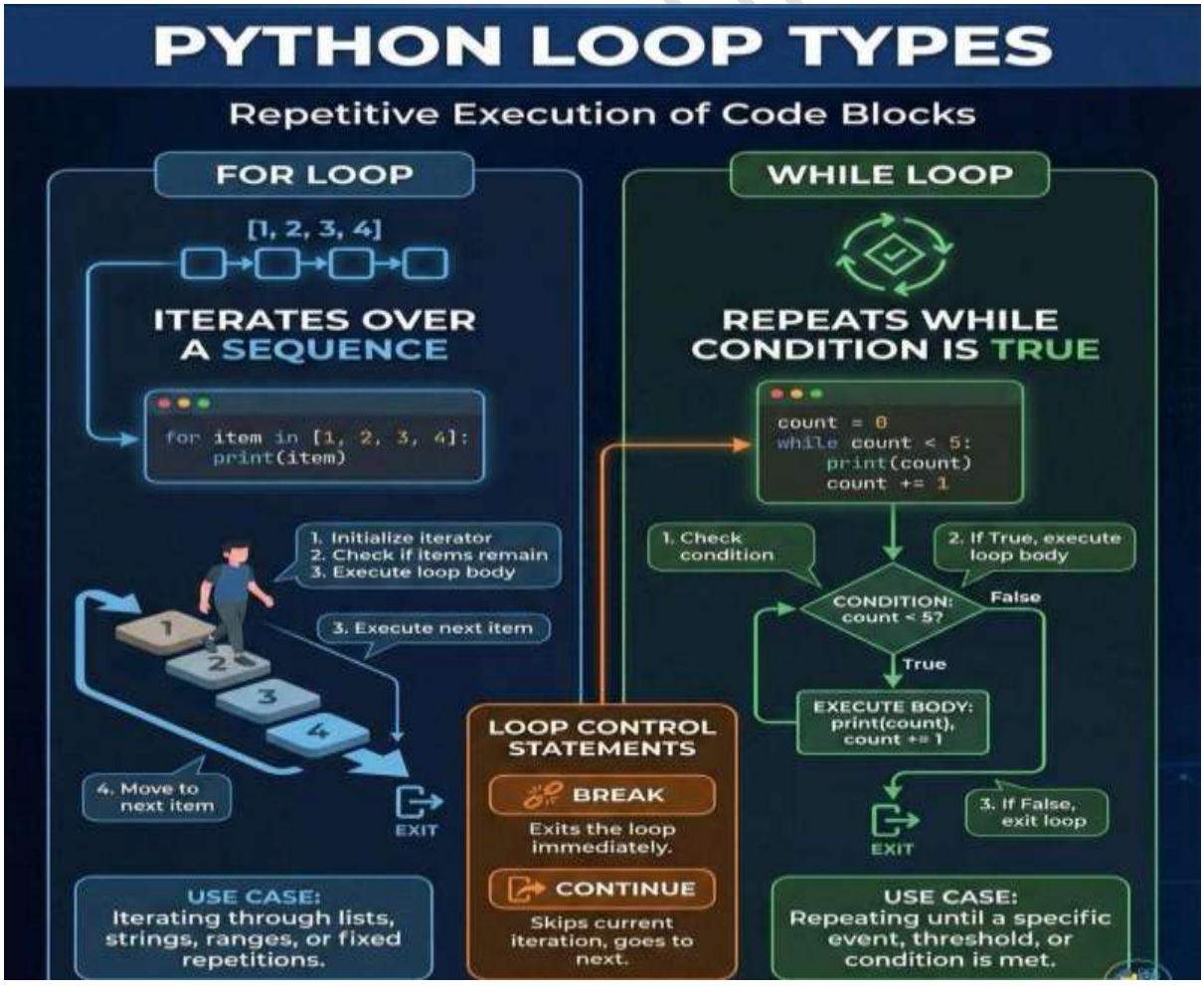
Difference Between Break and Continue

break	**continue**
Stops the loop completely.	Skips only the current iteration.
Control moves outside the loop.	Control moves to the next iteration.
Ends the loop early.	Loop continues running.

Break vs Continue Comparison

3DWEBSOFT FOUNDATION


Python Control Statements
Break Statement
Exit the loop entirely
Find item = 3
Item 1
Item 2
Item 3
1 for i in range(5):
2 if i == 3:
3 break
Continue Statement
Skip current iteration
Skip even numbers
Number 1
Number 3
Number 4
1 for i in range(5):
2 if i % 2 == 0:
3 continue


PYTHON LOOP TYPES
Repetitive Execution of Code Blocks
FOR LOOP
[1, 2, 3, 4]
ITERATES OVER A SEQUENCE
for item in [1, 2, 3, 4]:
print(item)
1. Initialize iterator
2. Check if items remain
3. Execute loop body
3. Execute next item
4. Move to next item
EXIT
USE CASE:
Iterating through lists, strings, ranges, or fixed repetitions.
WHILE LOOP
REPEATS WHILE CONDITION IS TRUE
count = 0
while count < 5:
print(count)
count += 1
1. Check condition
2. If True, execute loop body
CONDITION: count < 5?
False
True
EXECUTE BODY: print(count), count += 1
3. If False, exit loop
EXIT
USE CASE:
Repeating until a specific event, threshold, or condition is met.
LOOP CONTROL STATEMENTS
BREAK
Exits the loop immediately.
CONTINUE
Skips current iteration, goes to next.

Practical Session

Practical Activity 1

Stop at Number 7

```
for i in range(1, 11):
    if i == 7:
        break

    print(i)
```

Practical Activity 2

Skip Number 5

```
for i in range(1, 11):
    if i == 5:
        continue

    print(i)
```

Practical Activity 3

Print Even Numbers Only

```
for i in range(1, 11):
    if i % 2 != 0:
        continue

    print(i)
```

3DWEBSOFT FOUNDATION

Practical Activity 4

Stop When Password is Correct

```
while True:
    password = input("Enter Password: ")

    if password == "python123":
        print("Access Granted")
        break

    print("Wrong Password")
```

Practical Activity 5

Skip Multiples of 3

```
for i in range(1, 16):
    if i % 3 == 0:
        continue

    print(i)
```

Practical Activity 6

Search for a Number

```
numbers = [5, 8, 12, 18, 25]

for number in numbers:
```

3DWEBSOFT FOUNDATION

```
    if number == 18:
        print("Number Found!")
        break
```

Practical Activity 7

Countdown with Break

```
count = 10

while count >= 1:
    print(count)

    if count == 5:
        break

    count -= 1
```

Practical Activity 8

Skip a Fruit

```
fruits = ["Apple", "Banana", "Mango", "Orange"]

for fruit in fruits:
    if fruit == "Mango":
        continue
```

3DWEBSOFT FOUNDATION

```
print(fruit)
```

Classroom Coding Activity







Activity 1

Print numbers from **1 to 20**, but stop at **15** using break.

Activity 2

Print numbers from **1 to 20**, skipping **10** using continue.

Activity 3

Print only odd numbers from **1 to 20** using continue.

Activity 4

Create a password checker that repeats until the correct password is entered.

Real-Life Applications

Situation	Statement Used
Stop searching after finding a book	break
Skip an absent student during attendance	continue
Exit a game when the player quits	break
Skip invalid data while processing	continue
Stop checking after the correct password	break
Ignore damaged items during inspection	continue

Common Mistakes

❌ **Using break Outside a Loop**

Wrong

break

3DWEBSOFT FOUNDATION

Python will show an error because break can only be used **inside a loop**.

✅ **Correct**

```
for i in range(5):
    if i == 3:
        break
```

❌ **Forgetting to Update the Counter in a While Loop**

Wrong

```
count = 1

while count <= 5:
    if count == 3:
        continue

    print(count)
```

❌ This creates an **infinite loop** because count never changes.

✅ **Correct**

```
count = 1

while count <= 5:
    if count == 3:
```

3DWEBSOFT FOUNDATION

```
    count += 1
    continue

print(count)
count += 1
```

Best Practices

- ✓ Use break only when you need to stop the loop.
- ✓ Use continue to skip unnecessary steps.
- ✓ Update the counter variable in while loops.
- ✓ Write meaningful comments.
- ✓ Test your program with different inputs.

Keywords

- **Loop Control Statement** – Changes the normal flow of a loop.
- **Break** – Stops the loop immediately.
- **Continue** – Skips the current iteration.
- **Iteration** – One repetition of a loop.
- **Condition** – A True or False test.

Lesson Summary

- break immediately ends a loop.

3DWEBSOFT FOUNDATION

- continue skips the current iteration and continues with the next one.
- Both statements work with for and while loops.
- break is useful when the required result is found.
- continue is useful when some values need to be ignored.
- Loop control statements make programs more efficient and easier to understand.

Quiz

Multiple Choice Questions

1. Which statement stops a loop immediately?

A) continue

B) break

C) if

D) range

✅ **Answer: B**

2. Which statement skips only the current iteration?

A) break

B) continue

C) while

D) print

✅ **Answer: B**

3DWEBSOFT FOUNDATION

3. What will this code print?

```
for i in range(1, 6):
    if i == 4:
        break
    print(i)
```

A) 1 2 3

B) 1 2 3 4

C) 1 2 3 4 5

D) 4 5

✅ **Answer: A**

4. What will this code print?

```
for i in range(1, 6):
    if i == 2:
        continue
    print(i)
```

A) 1 2 3 4 5

B) 2 3 4 5

C) 1 3 4 5

D) 1 2 3 4

✅ **Answer: C**

5. Can break and continue be used outside a loop?

A) Yes

3DWEBSOFT FOUNDATION

B) No

✅ **Answer: B**

Homework

1. Write a program to print numbers from **1 to 30**, but stop at **20** using break.
2. Write a program to print numbers from **1 to 20**, skipping **7** using continue.
3. Write a program to print only even numbers from **1 to 20** using continue.
4. Create a password checker that repeats until the correct password is entered using a while loop and break.
5. Create a list of fruits and print all fruits except **Banana** using continue.
6. Write a program to search for a number in a list and stop when it is found using break.

3DWEBSOFT FOUNDATION